Übungsblatt 7

RESTful API-Design, HATEOAS & SSL/TLS

5430UE Web and Data Engineering

27. Mai 2026

Prof. Dr. Michael Granitzer, Tobias Fuchs

Lernziele

- Grundlegendes Verständnis der HTTP-Semantik zur Nutzung von Methoden und Status-Codes.
- Einblick in ressourcenorientiertes URL-Design und die Konzepte von HATEOAS.
- Kenntnis der wesentlichen Unterschiede zwischen REST- und SOAP-Architekturen.
- Basiswissen zur Transport-Sicherheit mittels TLS und dem Ablauf des Handshakes.

HTTP-Methoden und Status-Codes

HTTP-Methoden und Status-Codes(1/2)

In dieser Aufgabe betrachten wir die HTTP-Methoden POST, GET, PUT, PATCH, DELETE, HEAD, OPTIONS.

1. Beschreiben Sie den Zweck jeder der Methoden in einem Satz. Geben Sie jeweils an, ob die Methode sicher und/oder idempotent ist.

HTTP-Methoden und Status-Codes(2/2)

In dieser Aufgabe betrachten wir die HTTP-Methoden POST, GET, PUT, PATCH, DELETE, HEAD, OPTIONS.

2. Ordnen Sie jeder Situation die korrekte HTTP-Methode und passenden Status-Code zu:

Situation	Methode	Status-Code
Produktliste abrufen	?	?
Neues Produkt anlegen	?	?
Produktname aktualisieren (nur ein Feld)	?	?
Produkt vollständig ersetzen	?	?
Produkt löschen	?	?
Ressource nicht gefunden	-	?
Validierungsfehler im Request-Body	-	?
Server-interner Fehler	-	?

HTTP-Methoden und Status-Codes — Lösung (1/4)

1. HTTP-Methoden:

HTTP-Methoden und Status-Codes — Lösung (1/4)

1. HTTP-Methoden:

- **POST**: Schickt strukturierte Daten zur Verarbeitung an den Server, typischerweise zum Erzeugen einer neuen Ressource.
  Sicher: nein, idempotent: nein

HTTP-Methoden und Status-Codes — Lösung (1/4)

1. HTTP-Methoden:

- **POST**: Schickt strukturierte Daten zur Verarbeitung an den Server, typischerweise zum Erzeugen einer neuen Ressource.
  Sicher: nein, idempotent: nein
- **GET**: Dient zum Anfordern und Auslesen einer Ressource vom Webserver in verschiedenen Formaten wie JSON, ohne dabei Daten zu verändern.
  Sicher: ja, idempotent: ja

HTTP-Methoden und Status-Codes — Lösung (1/4)

1. HTTP-Methoden:

- **POST**: Schickt strukturierte Daten zur Verarbeitung an den Server, typischerweise zum Erzeugen einer neuen Ressource.
  Sicher: nein, idempotent: nein
- **GET**: Dient zum Anfordern und Auslesen einer Ressource vom Webserver in verschiedenen Formaten wie JSON, ohne dabei Daten zu verändern.
  Sicher: ja, idempotent: ja
- **PUT**: Lädt eine Ressource auf den Webserver hoch, um eine Ressource zu ersetzen oder sie legt unter einer bekannten URI neu anzulegen.
  Sicher: nein, idempotent: ja

HTTP-Methoden und Status-Codes — Lösung (1/4)

1. HTTP-Methoden:

- **POST**: Schickt strukturierte Daten zur Verarbeitung an den Server, typischerweise zum Erzeugen einer neuen Ressource.
  Sicher: nein, idempotent: nein
- **GET**: Dient zum Anfordern und Auslesen einer Ressource vom Webserver in verschiedenen Formaten wie JSON, ohne dabei Daten zu verändern.
  Sicher: ja, idempotent: ja
- **PUT**: Lädt eine Ressource auf den Webserver hoch, um eine Ressource zu ersetzen oder sie legt unter einer bekannten URI neu anzulegen.
  Sicher: nein, idempotent: ja
- **PATCH**: Führt eine partielle Änderung (Teil-Update) an einer bestehenden Ressource durch, wobei oft spezielle Änderungsanweisungen statt des gesamten Objekts gesendet werden.
  Sicher: nein, idempotent: abhängig von der Implementierung, meist nein

HTTP-Methoden und Status-Codes — Lösung (2/4)



HTTP-Methoden und Status-Codes — Lösung (2/4)

- **DELETE**: Dient zum gezielten Löschen von spezifischen Ressourcen auf dem Server.
  Sicher: nein, idempotent: ja

HTTP-Methoden und Status-Codes — Lösung (2/4)

- **DELETE**: Dient zum gezielten Löschen von spezifischen Ressourcen auf dem Server.
  Sicher: nein, idempotent: ja
- **HEAD**: Entspricht GET, liefert jedoch nur die Header-Informationen ohne Response-Body zurück.
  Sicher: ja, idempotent: ja

HTTP-Methoden und Status-Codes — Lösung (2/4)

- **DELETE**: Dient zum gezielten Löschen von spezifischen Ressourcen auf dem Server.
  Sicher: nein, idempotent: ja
- **HEAD**: Entspricht GET, liefert jedoch nur die Header-Informationen ohne Response-Body zurück.
  Sicher: ja, idempotent: ja
- **OPTIONS**: Erlaubt die Anfrage, welche HTTP-Methoden oder Kommunikationsoptionen für eine Ressource unterstützt werden.
  Sicher: ja, idempotent: ja

HTTP-Methoden und Status-Codes — Lösung (2/4)

- **DELETE**: Dient zum gezielten Löschen von spezifischen Ressourcen auf dem Server.
  Sicher: nein, idempotent: ja
- **HEAD**: Entspricht GET, liefert jedoch nur die Header-Informationen ohne Response-Body zurück.
  Sicher: ja, idempotent: ja
- **OPTIONS**: Erlaubt die Anfrage, welche HTTP-Methoden oder Kommunikationsoptionen für eine Ressource unterstützt werden.
  Sicher: ja, idempotent: ja

Zusatz: *Sicher (Safe):* Methode verändert keine Ressource — kann ohne Nebenwirkungen wiederholt werden. CDNs cachen nur safe Methoden.
Idempotent: Mehrfache Ausführung hat denselben Effekt wie einmal — wichtig für Retry-Logik in Load Balancern und API-Gateways.

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen		
Neues Produkt anlegen		
Produktname aktualisieren (nur ein Feld)		
Produkt vollständig ersetzen		
Produkt löschen		
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	
Neues Produkt anlegen		
Produktname aktualisieren (nur ein Feld)		
Produkt vollständig ersetzen		
Produkt löschen		
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen		
Produktname aktualisieren (nur ein Feld)		
Produkt vollständig ersetzen		
Produkt löschen		
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	
Produktname aktualisieren (nur ein Feld)		
Produkt vollständig ersetzen		
Produkt löschen		
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)		
Produkt vollständig ersetzen		
Produkt löschen		
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	
Produkt vollständig ersetzen		
Produkt löschen		
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen		
Produkt löschen		
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	
Produkt löschen		
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen		
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen	DELETE	
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen	DELETE	204 No Content
Ressource nicht gefunden		
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen	DELETE	204 No Content
Ressource nicht gefunden	-	
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen	DELETE	204 No Content
Ressource nicht gefunden	-	404 Not Found
Validierungsfehler im Request-Body		
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen	DELETE	204 No Content
Ressource nicht gefunden	-	404 Not Found
Validierungsfehler im Request-Body	-	
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen	DELETE	204 No Content
Ressource nicht gefunden	-	404 Not Found
Validierungsfehler im Request-Body	-	400 Bad Request
Server-interner Fehler		

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen	DELETE	204 No Content
Ressource nicht gefunden	-	404 Not Found
Validierungsfehler im Request-Body	-	400 Bad Request
Server-interner Fehler	-	

HTTP-Methoden und Status-Codes — Lösung (3/4)

2. Zuordnungen:

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (nur ein Feld)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen	DELETE	204 No Content
Ressource nicht gefunden	-	404 Not Found
Validierungsfehler im Request-Body	-	400 Bad Request
Server-interner Fehler	-	500 Internal Server Error

HTTP-Methoden und Status-Codes — Lösung (4/4)

Zusatz: Statuscode-Bereiche

- **1xx**: Informationelle Antworten
- **2xx**: Erfolgreiche Requests
- **3xx**: Weiterleitungen (Redirects)
- **4xx**: Client-Fehler
- **5xx**: Server-Fehler

Caching-Strategien

Caching-Strategien

Ein Online-Shop liefert verschiedene Ressourcen. Bestimmen Sie für jede die geeignete Caching-Strategie:

Ressource	Strategie (kein Cache / kurz / lang / ewig)	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	?	?
Produktliste (aktualisiert sich stündlich)	?	?
Warenkorb-Inhalt (nutzerspezifisch)	?	?
JavaScript-Bundle `app.abc123.js` (Hash im Namen)	?	?

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)		
Produktliste (aktualisiert sich stündlich)		
Warenkorb-Inhalt (nutzerspezifisch)		
JavaScript-Bundle `app.abc123.js` (Hash im Namen)		

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	Ewig	
Produktliste (aktualisiert sich stündlich)		
Warenkorb-Inhalt (nutzerspezifisch)		
JavaScript-Bundle `app.abc123.js` (Hash im Namen)		

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`
Produktliste (aktualisiert sich stündlich)		
Warenkorb-Inhalt (nutzerspezifisch)		
JavaScript-Bundle `app.abc123.js` (Hash im Namen)		

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`
Produktliste (aktualisiert sich stündlich)	Kurz	
Warenkorb-Inhalt (nutzerspezifisch)		
JavaScript-Bundle `app.abc123.js` (Hash im Namen)		

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`
Produktliste (aktualisiert sich stündlich)	Kurz	`Cache-Control: public, max-age=3600`
Warenkorb-Inhalt (nutzerspezifisch)		
JavaScript-Bundle `app.abc123.js` (Hash im Namen)		

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`
Produktliste (aktualisiert sich stündlich)	Kurz	`Cache-Control: public, max-age=3600`
Warenkorb-Inhalt (nutzerspezifisch)	Kein Cache	
JavaScript-Bundle `app.abc123.js` (Hash im Namen)		

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`
Produktliste (aktualisiert sich stündlich)	Kurz	`Cache-Control: public, max-age=3600`
Warenkorb-Inhalt (nutzerspezifisch)	Kein Cache	`Cache-Control: private, no-store`
JavaScript-Bundle `app.abc123.js` (Hash im Namen)		

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`
Produktliste (aktualisiert sich stündlich)	Kurz	`Cache-Control: public, max-age=3600`
Warenkorb-Inhalt (nutzerspezifisch)	Kein Cache	`Cache-Control: private, no-store`
JavaScript-Bundle `app.abc123.js` (Hash im Namen)	Ewig	

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`
Produktliste (aktualisiert sich stündlich)	Kurz	`Cache-Control: public, max-age=3600`
Warenkorb-Inhalt (nutzerspezifisch)	Kein Cache	`Cache-Control: private, no-store`
JavaScript-Bundle `app.abc123.js` (Hash im Namen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)

Caching-Strategien — Lösung (1/3)

Ressource	Strategie	HTTP-Header
Produktbild logo.png (unveränderlich, mit Hash im Dateinamen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`
Produktliste (aktualisiert sich stündlich)	Kurz	`Cache-Control: public, max-age=3600`
Warenkorb-Inhalt (nutzerspezifisch)	Kein Cache	`Cache-Control: private, no-store`
JavaScript-Bundle `app.abc123.js` (Hash im Namen)	Ewig	`Cache-Control: public, max-age=31536000, immutable`

Anmerkungen:

- Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden → Änderung = neuer Hash = neue URL.
- Nutzerspezifische Daten nie öffentlich cachen (private)
- Zur Warenkorbaufgabe: no-store verhindert, dass der Browser bei der Navigation (z. B. über den „Zurück“-Button) eine veraltete Kopie des Einkaufswagens anzeigt. Es zwingt den Client, bei jedem Seitenaufruf den echten, aktuellen Zustand live vom Server abzufragen.

Caching-Strategien — Lösung (2/3)

Zusatz: Wichtige Cache-Header und Attribute

Header / Attribut	Erklärung
`Cache-Control`	Zentraler Header zur Steuerung des HTTP-Cachings; definiert Regeln für Browser, CDNs und Proxies.
`public`	Ressource darf von gemeinsamen Caches (CDN, Proxy) gespeichert werden.
`private`	Ressource darf nur im Browser-Cache des Nutzers gespeichert werden, nicht in Shared Caches.
`no-store`	Ressource darf überhaupt nicht gespeichert werden (weder Browser noch Proxy/CDN).
`no-cache`	Cache darf speichern, muss die Ressource aber vor Wiederverwendung beim Server validieren.
`max-age=<Sekunden>`	Gibt die maximale Lebensdauer im Cache an, z.B. `max-age=3600` für eine Stunde.

Caching-Strategien — Lösung (3/3)

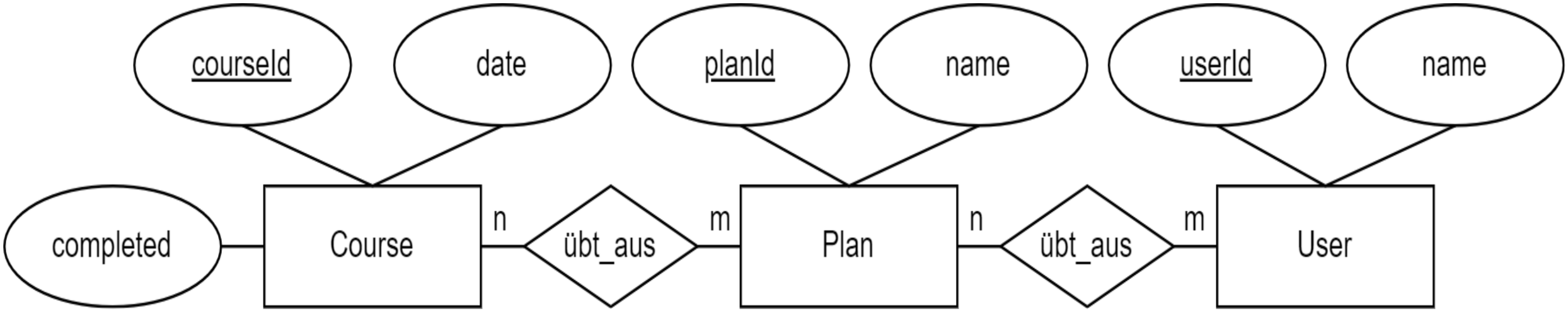
Zusatz: Wichtige Cache-Header und Attribute

Header / Attribut	Erklärung
`immutable`	Signalisiert, dass sich die Ressource nicht verändert; sinnvoll bei Dateien mit Content-Hash im Dateinamen.
`ETag`	Eindeutiger Versionsbezeichner einer Ressource; ermöglicht effiziente Cache-Validierung.
`Last-Modified`	Zeitpunkt der letzten Änderung einer Ressource.
`Age`	Zeigt an, wie lange sich eine Response bereits in einem Cache befindet.

RESTful URL-Design

RESTful URL-Design (1/3)

Gefordert ist der Entwurf eines RESTful WebService zur Verwaltung von Trainingsplänen eines Fitnesscenters. Nehmen Sie dazu folgendes (vereinfachtes) Datenschema an:



Ein Trainingsplan beinhaltet mehrere Übungen, und mehrere Personen können denselben Trainingsplan ausüben.

RESTful URL-Design (2/3)

Ein Backend soll folgende Operationen unterstützen:

1. Alle Trainingspläne eines Nutzers abrufen
2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen
3. Alle Personen anzeigen, die einen gewissen Namen haben
4. Alle abgeschlossen Kurse eines Nutzers an einen spezifischen Tag filtern
5. Einen neuen Trainingsplan für einen Nutzer erstellen
6. Einen Kurs in einem Trainingsplan aktualisieren
7. Einen Kurs mit gegebener Id löschen



RESTful URL-Design (3/3)

1. Entwerfen Sie ein URL Mapping für einen RESTful WebService für die oben genannten 7 Funktionen. Bestimmen Sie jeweils die HTTP Methode und URL, die ein Client aufzurufen hat, um die Funktion auszuführen. Halten Sie sich für das URL Design sowie die Wahl der HTTP Methoden an die Konventionen aus der Vorlesung. Kennzeichnen Sie dynamische Pfadelemente mit geschweiften Klammern ({}). Die Angabe von Protokoll und Domain können Sie mit "fitness.de" abkürzen. Sollten für eine Anfrage etwaige Parameter mit übertragen werden müssen, geben Sie nur Parameter in der URL an (der Inhalt des Request Body muss nicht mit angegeben werden).
2. Welche der URL-Muster sind dabei problematisch:
   - `/getTrainings`
   - `/user/deleteById?id=5`



RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:

RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

```
GET fitness.de/users/{userId}/plans
```

RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

```
GET fitness.de/users/{userId}/plans
```

    2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen:

RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

```
GET fitness.de/users/{userId}/plans
```

    2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen:

```
GET fitness.de/users/{userId}/plans/{planId}/courses/{courseId}
```

RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

```
GET fitness.de/users/{userId}/plans
```

    2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen:

```
GET fitness.de/users/{userId}/plans/{planId}/courses/{courseId}
```

    3. Alle Personen anzeigen, die einen gewissen Namen haben:



RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

```
GET fitness.de/users/{userId}/plans
```

    2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen:

```
GET fitness.de/users/{userId}/plans/{planId}/courses/{courseId}
```

    3. Alle Personen anzeigen, die einen gewissen Namen haben:

```
GET fitness.de/users?name=x
```



RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

```
GET fitness.de/users/{userId}/plans
```

    2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen:

```
GET fitness.de/users/{userId}/plans/{planId}/courses/{courseId}
```

    3. Alle Personen anzeigen, die einen gewissen Namen haben:

```
GET fitness.de/users?name=x
```

    4. Alle abgeschlossen Kurse eines Nutzers an einen spezifischen Tag filtern:



RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

```
GET fitness.de/users/{userId}/plans
```

    2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen:

```
GET fitness.de/users/{userId}/plans/{planId}/courses/{courseId}
```

    3. Alle Personen anzeigen, die einen gewissen Namen haben:

```
GET fitness.de/users?name=x
```

    4. Alle abgeschlossen Kurse eines Nutzers an einen spezifischen Tag filtern:

```
GET fitness.de/users/{userId}/courses?date=x&completed=true
```

RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

```
GET fitness.de/users/{userId}/plans
```

    2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen:

```
GET fitness.de/users/{userId}/plans/{planId}/courses/{courseId}
```

    3. Alle Personen anzeigen, die einen gewissen Namen haben:

```
GET fitness.de/users?name=x
```

    4. Alle abgeschlossen Kurse eines Nutzers an einen spezifischen Tag filtern:

```
GET fitness.de/users/{userId}/courses?date=x&completed=true
```

    5. Einen neuen Trainingsplan für einen Nutzer erstellen:



RESTful URL-Design — Lösung (1/4)

1. URL-Mappings:
    1. Alle Trainingspläne eines Nutzers abrufen:

```
GET fitness.de/users/{userId}/plans
```

    2. Einen spezifischen Kurs aus dem Trainingsplan einer Person abrufen:

```
GET fitness.de/users/{userId}/plans/{planId}/courses/{courseId}
```

    3. Alle Personen anzeigen, die einen gewissen Namen haben:

```
GET fitness.de/users?name=x
```

    4. Alle abgeschlossen Kurse eines Nutzers an einen spezifischen Tag filtern:

```
GET fitness.de/users/{userId}/courses?date=x&completed=true
```

    5. Einen neuen Trainingsplan für einen Nutzer erstellen:

```
POST fitness.de/users/{userId}/plans
```



RESTful URL-Design — Lösung (2/4)



RESTful URL-Design — Lösung (2/4)

6. Einen Kurs in einem Trainingsplan aktualisieren:

RESTful URL-Design — Lösung (2/4)

6. Einen Kurs in einem Trainingsplan aktualisieren:

```
PUT fitness.de/users/{userId}/courses/{courseId}
```



RESTful URL-Design — Lösung (2/4)

6. Einen Kurs in einem Trainingsplan aktualisieren:

```
PUT fitness.de/users/{userId}/courses/{courseId}
```

7. Einen Kurs mit gegebener Id löschen:

RESTful URL-Design — Lösung (2/4)

6. Einen Kurs in einem Trainingsplan aktualisieren:

```
PUT fitness.de/users/{userId}/courses/{courseId}
```

7. Einen Kurs mit gegebener Id löschen:

```
DELETE fitness.de/courses/{courseId}
```



RESTful URL-Design — Lösung (3/4)

2. Problematische Muster:

- `/getTrainings`:
-

RESTful URL-Design — Lösung (3/4)

2. Problematische Muster:

- `/getTrainings`:

  Verb in der URL verletzt REST: HTTP-Methoden (GET, POST, …) sind die Verben, URLs sind Substantive (Ressourcen).

-

RESTful URL-Design — Lösung (3/4)

2. Problematische Muster:

- `/getTrainings`:

  Verb in der URL verletzt REST: HTTP-Methoden (GET, POST, …) sind die Verben, URLs sind Substantive (Ressourcen).

- `/user/deleteById?id=5`:

RESTful URL-Design — Lösung (3/4)

2. Problematische Muster:

- `/getTrainings`:

  Verb in der URL verletzt REST: HTTP-Methoden (GET, POST, …) sind die Verben, URLs sind Substantive (Ressourcen).

- `/user/deleteById?id=5`:

  Operation im Pfad und Query-Parameter für ID, Plural für Collection; korrekt: `DELETE /users/5`.

RESTful URL-Design — Lösung (4/4)

Zusatz: RESTful URI-Design-Regeln

Regel	Erklärung / Beispiel
Plural für Collections	Ressourcen-Sammlungen werden im Plural benannt, da sie mehrere Elemente enthalten. Beispiel: `/users`, `/products`, `/orders`.
Kleinschreibung und Bindestriche	URLs verwenden üblicherweise Kleinbuchstaben und Bindestriche für bessere Lesbarkeit und Konsistenz. Beispiel: `/order-items`.
Keine Verben im Pfad	Die HTTP-Methode beschreibt die Aktion (GET, POST, PUT, DELETE); die URL beschreibt nur die Ressource. Beispiel: `POST /orders` statt `/createOrder`.
Hierarchie drückt Zugehörigkeit aus	Verschachtelte Pfade modellieren Beziehungen zwischen Ressourcen. Beispiel: `/users/{userId}/orders`.



Regel	Erklärung / Beispiel
Pfad = Identität, Query = Parameter	Der Pfad identifiziert Ressourcen, Query-Parameter dienen zum Filtern, Suchen, Sortieren oder für optionale Angaben. Beispiele: `/users/{id}`, `/products?sort=price`, `/users?name=Max`.

HATEOAS und API-Versionierung

HATEOAS und API-Versionierung

1. **HATEOAS**:
    - Was bedeutet HATEOAS?
    - Welchen praktischen Vorteil bietet es gegenüber einer reinen Daten-API?
    - Zeigen Sie ein JSON-Beispiel für eine Produktressource mit HATEOAS-Links.
2. Nennen Sie zwei Strategien für **API-Versionierung** und diskutieren Sie je einen Vorteil und Nachteil.

HATEOAS und API-Versionierung — Lösung (1/4)

1. **HATEOAS**:
   - HATEOAS (Hypermedia as the Engine of Application State) ist eine Anforderung an eine REST-API. Es besagt, dass ein Client nicht vorab wissen muss, welche URLs für welche Aktionen existieren. Stattdessen liefert der Server mit jeder Antwort die aktuell erlaubten Navigationsmöglichkeiten in Form von Hyperlinks mit.
   - Die Struktur der API-Adressen (URLs) kann auf dem Server jederzeit verändert oder verschoben werden, ohne dass die Client-Anwendung ihre Funktionsfähigkeit verliert. Da der Client nur den Namen der Aktion kennt und dem jeweils aktuell mitgelieferten Link folgt, findet er die Ressource auch unter einer neuen Adresse automatisch wieder.

HATEOAS und API-Versionierung — Lösung (2/4)

- Beispiel:

```
{
 "id": 42,
 "name": "Laptop Pro",
 "price": 999.00,
 "_links": {
   "self": { "href": "/products/42" },
   "reviews": { "href": "/products/42/reviews" },
   "order": { "href": "/orders" }
 }
}
```

HATEOAS und API-Versionierung — Lösung (3/4)

2. API-Versionierung:

Strategie	Vorteil	Nachteil

HATEOAS und API-Versionierung — Lösung (3/4)

2. API-Versionierung:

Strategie	Vorteil	Nachteil
URL-Versionierung (Versionsnummer direkt in der Adresse) `/api/v1/products`		

HATEOAS und API-Versionierung — Lösung (3/4)

2. API-Versionierung:

Strategie	Vorteil	Nachteil
URL-Versionierung (Versionsnummer direkt in der Adresse) `/api/v1/products`	Explizit, einfach zu verstehen und zu testen	

HATEOAS und API-Versionierung — Lösung (3/4)

2. API-Versionierung:

Strategie	Vorteil	Nachteil
URL-Versionierung (Versionsnummer direkt in der Adresse) `/api/v1/products`	Explizit, einfach zu verstehen und zu testen	Dupliziert Routen; Clients müssen URL-Änderung mitmachen

HATEOAS und API-Versionierung — Lösung (3/4)

2. API-Versionierung:

Strategie	Vorteil	Nachteil
URL-Versionierung (Versionsnummer direkt in der Adresse) `/api/v1/products`	Explizit, einfach zu verstehen und zu testen	Dupliziert Routen; Clients müssen URL-Änderung mitmachen
Header-Versionierung (URL bleibt unverändert, z.B. `https://api.shop.de/products`. Die gewünschte API-Version wird über den HTTP-`Accept`-Header übertragen.) `Accept: application/vnd.shop.v2+json`		

HATEOAS und API-Versionierung — Lösung (3/4)

2. API-Versionierung:

Strategie	Vorteil	Nachteil
URL-Versionierung (Versionsnummer direkt in der Adresse) `/api/v1/products`	Explizit, einfach zu verstehen und zu testen	Dupliziert Routen; Clients müssen URL-Änderung mitmachen
Header-Versionierung (URL bleibt unverändert, z.B. `https://api.shop.de/products`. Die gewünschte API-Version wird über den HTTP-`Accept`-Header übertragen.) `Accept: application/vnd.shop.v2+json`	URL bleibt stabil; Versionierung ist von der Ressourcenadresse getrennt und nutzt HTTP-Content-Negotiation.	

HATEOAS und API-Versionierung — Lösung (3/4)

2. API-Versionierung:

Strategie	Vorteil	Nachteil
URL-Versionierung (Versionsnummer direkt in der Adresse) `/api/v1/products`	Explizit, einfach zu verstehen und zu testen	Dupliziert Routen; Clients müssen URL-Änderung mitmachen
Header-Versionierung (URL bleibt unverändert, z.B. `https://api.shop.de/products`. Die gewünschte API-Version wird über den HTTP-`Accept`-Header übertragen.) `Accept: application/vnd.shop.v2+json`	URL bleibt stabil; Versionierung ist von der Ressourcenadresse getrennt und nutzt HTTP-Content-Negotiation.	Weniger sichtbar und schwerer direkt zu testen, da die Version nicht in der URL, sondern im Header steckt. Debugging, Logging, Routing und Caching werden komplexer.

HATEOAS und API-Versionierung — Lösung (4/4)

2. API-Versionierung:

HATEOAS und API-Versionierung — Lösung (4/4)

2. API-Versionierung:

Anmerkung:

HATEOAS und API-Versionierung — Lösung (4/4)

2. API-Versionierung:

Anmerkung:

- **Bisherige Gefahr (Statische Kopplung):** Der Client sucht starr nach einer fest einprogrammierten Adresse. Wenn der Server diese Struktur ändert, läuft die Anfrage ins Leere.

HATEOAS und API-Versionierung — Lösung (4/4)

2. API-Versionierung:

Anmerkung:

- **Bisherige Gefahr (Statische Kopplung):** Der Client sucht starr nach einer fest einprogrammierten Adresse. Wenn der Server diese Struktur ändert, läuft die Anfrage ins Leere.
- **Lösung durch HATEOAS (Dynamische Navigation):** Der Client orientiert sich nur am Namen einer Aktion. Der Server liefert die aktuell gültige URL bei jeder Antwort mit. Der Client folgt einfach dem aktuell bereitgestellten „Wegweiser“.

SOAP vs REST

SOAP vs REST

1. Erklären Sie REST und SOAP kurz in jeweils 1-2 Sätzen.
2. Entscheiden Sie für jedes Szenario, ob REST oder SOAP besser geeignet ist, und begründen Sie:

Szenario	REST oder SOAP	Begründung
Mobile App ruft Produktdaten ab	?	?
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)	?	?
IoT-Sensoren senden Messwerte (geringe Bandbreite)	?	?
Legacy-Enterprise-Integration mit strenger Vertragsbindung	?	?

SOAP vs REST — Lösung (1/2)

1. **REST**: REST ist ein ressourcenorientierter Architekturstil, der die vorhandenen Mechanismen des HTTP-Protokolls (wie GET oder POST) nutzt, um Ressourcen effizient zu verwalten. Er ist der moderne Standard für Web- und Mobile-Apps, da er durch das JSON-Format sehr schnell, einfach zu implementieren und hochgradig skalierbar ist.

   SOAP (Simple Object Access Protocol): SOAP ist ein XML-basiertes Nachrichtenprotokoll, das auf festen Standards und formalen Verträgen (WSDL) beruht. Es wird vor allem in hochsicheren Enterprise-Umgebungen eingesetzt, da es komplexe Transaktionen und eine starke Typisierung garantiert.

   Entscheidung: REST wird bevorzugt, wenn eine performante, leichtgewichtige und flexibel konsumierbare API benötigt wird. SOAP eignet sich besonders für Enterprise- und Integrationsszenarien, in denen formale Schnittstellenverträge (WSDL), starke Standardisierung, Sicherheit oder Transaktionsunterstützung erforderlich sind.

SOAP vs REST — Lösung (2/2)

2. REST vs. SOAP:

Szenario	REST/SOAP	Begründung
Mobile App ruft Produktdaten ab		
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)		
IoT-Sensoren senden Messwerte (geringe Bandbreite)		
Legacy-Enterprise-Integration mit strenger Vertragsbindung		



SOAP vs REST — Lösung (2/2)

2. REST vs. SOAP:

Szenario	REST/SOAP	Begründung
Mobile App ruft Produktdaten ab	REST	
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)		
IoT-Sensoren senden Messwerte (geringe Bandbreite)		
Legacy-Enterprise-Integration mit strenger Vertragsbindung		

SOAP vs REST — Lösung (2/2)

2. REST vs. SOAP:

Szenario	REST/SOAP	Begründung
Mobile App ruft Produktdaten ab	REST	Leichtgewichtig, JSON, HTTP-Caching, geringe Latenz
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)		
IoT-Sensoren senden Messwerte (geringe Bandbreite)		
Legacy-Enterprise-Integration mit strenger Vertragsbindung		

SOAP vs REST — Lösung (2/2)

2. REST vs. SOAP:

Szenario	REST/SOAP	Begründung
Mobile App ruft Produktdaten ab	REST	Leichtgewichtig, JSON, HTTP-Caching, geringe Latenz
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)	SOAP	
IoT-Sensoren senden Messwerte (geringe Bandbreite)		
Legacy-Enterprise-Integration mit strenger Vertragsbindung		

SOAP vs REST — Lösung (2/2)

2. REST vs. SOAP:

Szenario	REST/SOAP	Begründung
Mobile App ruft Produktdaten ab	REST	Leichtgewichtig, JSON, HTTP-Caching, geringe Latenz
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)	SOAP	Bei Finanztransaktionen steht nicht die Geschwindigkeit, sondern die absolute Datensicherheit und Korrektheit (ACID-Eigenschaften) im Vordergrund, SOAP unterstützt über WS-*-Standards sichere, transaktionale Enterprise-Kommunikation.
IoT-Sensoren senden Messwerte (geringe Bandbreite)		
Legacy-Enterprise-Integration mit strenger Vertragsbindung		

SOAP vs REST — Lösung (2/2)

2. REST vs. SOAP:

Szenario	REST/SOAP	Begründung
Mobile App ruft Produktdaten ab	REST	Leichtgewichtig, JSON, HTTP-Caching, geringe Latenz
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)	SOAP	Bei Finanztransaktionen steht nicht die Geschwindigkeit, sondern die absolute Datensicherheit und Korrektheit (ACID-Eigenschaften) im Vordergrund, SOAP unterstützt über WS-*-Standards sichere, transaktionale Enterprise-Kommunikation.
IoT-Sensoren senden Messwerte (geringe Bandbreite)	REST	
Legacy-Enterprise-Integration mit strenger Vertragsbindung		

SOAP vs REST — Lösung (2/2)

2. REST vs. SOAP:

Szenario	REST/SOAP	Begründung
Mobile App ruft Produktdaten ab	REST	Leichtgewichtig, JSON, HTTP-Caching, geringe Latenz
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)	SOAP	Bei Finanztransaktionen steht nicht die Geschwindigkeit, sondern die absolute Datensicherheit und Korrektheit (ACID-Eigenschaften) im Vordergrund, SOAP unterstützt über WS-*-Standards sichere, transaktionale Enterprise-Kommunikation.
IoT-Sensoren senden Messwerte (geringe Bandbreite)	REST	Minimaler Overhead, JSON oder binäre Formate; SOAP-XML zu schwer
Legacy-Enterprise-Integration mit strenger Vertragsbindung		

SOAP vs REST — Lösung (2/2)

2. REST vs. SOAP:

Szenario	REST/SOAP	Begründung
Mobile App ruft Produktdaten ab	REST	Leichtgewichtig, JSON, HTTP-Caching, geringe Latenz
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)	SOAP	Bei Finanztransaktionen steht nicht die Geschwindigkeit, sondern die absolute Datensicherheit und Korrektheit (ACID-Eigenschaften) im Vordergrund, SOAP unterstützt über WS-*-Standards sichere, transaktionale Enterprise-Kommunikation.
IoT-Sensoren senden Messwerte (geringe Bandbreite)	REST	Minimaler Overhead, JSON oder binäre Formate; SOAP-XML zu schwer
Legacy-Enterprise-Integration mit strenger Vertragsbindung	SOAP	

SOAP vs REST — Lösung (2/2)

2. REST vs. SOAP:

Szenario	REST/SOAP	Begründung
Mobile App ruft Produktdaten ab	REST	Leichtgewichtig, JSON, HTTP-Caching, geringe Latenz
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)	SOAP	Bei Finanztransaktionen steht nicht die Geschwindigkeit, sondern die absolute Datensicherheit und Korrektheit (ACID-Eigenschaften) im Vordergrund, SOAP unterstützt über WS-*-Standards sichere, transaktionale Enterprise-Kommunikation.
IoT-Sensoren senden Messwerte (geringe Bandbreite)	REST	Minimaler Overhead, JSON oder binäre Formate; SOAP-XML zu schwer
Legacy-Enterprise-Integration mit strenger Vertragsbindung	SOAP	In großen, gewachsenen Unternehmensstrukturen müssen oft Systeme verschiedener Hersteller miteinander kommunizieren, die über Jahrzehnte stabil bleiben sollen; bestehende SOAP-Infrastruktur verwenden

TLS/SSL - Transport-Sicherheit

TLS/SSL - Transport-Sicherheit

1. Was war das zentrale Sicherheitsproblem von SSL 2.0, das zur Entwicklung von TLS führte?
2. Ist TLS heute für Web-Anwendungen verpflichtend oder optional? Begründen Sie aus technischer und regulatorischer Sicht.
3. Erklären Sie den TLS-Handshake in 4 Schritten: Was wird ausgetauscht und welches Ziel hat jeder Schritt?
4. Was bedeutet Certificate Pinning und für welche Art von Anwendung ist es besonders relevant?

TLS/SSL - Transport-Sicherheit — Lösung (1/4)

1. Schwachstellen von SSL 2.0:
   - SSL 2.0 verwendet veraltete kryptographische Verfahren (u.a. MD5) sowie schwache Cipher Suites, wodurch Integrität und Sicherheit der Kommunikation beeinträchtigt werden können.
   - Teile des Handshakes und die Aushandlung der Sicherheitsparameter sind unzureichend geschützt, sodass Angreifer Client und Server zu einer schwächeren Cipher Suite zwingen können (Downgrade-Angriff).
   - Verschlüsselung und Integritätsschutz sind kryptographisch nicht sauber getrennt; zusätzlich weist das Schlüsselmanagement konzeptionelle Schwächen auf.
   - Sitzungen können durch unzureichend geschützte Kontrollnachrichten beendet oder beeinflusst werden.

TLS/SSL - Transport-Sicherheit — Lösung (2/4)

2. Heute TLS:
   - **Technisch:** Moderne Webstandards und Browser setzen zunehmend auf HTTPS, indem unsichere HTTP-Seiten als „nicht sicher“ gekennzeichnet und bestimmte Funktionen nur noch über TLS unterstützt werden. → TLS faktisch notwendig für zuverlässige/vertrauenswürdige Bereitstellung.
   - **Regulatorisch:** Datenschutzvorgaben wie die DSGVO verlangen angemessene technische Maßnahmen zum Schutz personenbezogener Daten, wozu auch Verschlüsselung gehört. Zusätzlich schreiben Standards wie PCI-DSS den Einsatz von TLS bei sensiblen Daten verbindlich vor.

TLS/SSL - Transport-Sicherheit — Lösung (3/4)

3. TLS-Handshake in 4 Schritten:

TLS/SSL - Transport-Sicherheit — Lösung (3/4)

3. TLS-Handshake in 4 Schritten:
    1. **ClientHello:** Client sendet unterstützte TLS-Versionen und Cipher Suites an den Server.
       Ziel: Aushandlung kompatibler Sicherheitsparameter.

TLS/SSL - Transport-Sicherheit — Lösung (3/4)

3. TLS-Handshake in 4 Schritten:
    1. **ClientHello:** Client sendet unterstützte TLS-Versionen und Cipher Suites an den Server.
       Ziel: Aushandlung kompatibler Sicherheitsparameter.
    2. **ServerHello + Certificate:** Server wählt eine Cipher Suite aus und sendet sein X.509-Zertifikat.
       Ziel: Festlegung der Verschlüsselung und Authentifizierung des Servers.

TLS/SSL - Transport-Sicherheit — Lösung (3/4)

3. TLS-Handshake in 4 Schritten:
    1. **ClientHello:** Client sendet unterstützte TLS-Versionen und Cipher Suites an den Server.
       Ziel: Aushandlung kompatibler Sicherheitsparameter.
    2. **ServerHello + Certificate:** Server wählt eine Cipher Suite aus und sendet sein X.509-Zertifikat.
       Ziel: Festlegung der Verschlüsselung und Authentifizierung des Servers.
    3. **Key Exchange:** Beide Seiten führen einen Schlüsselaustausch durch und berechnen einen gemeinsamen Sitzungsschlüssel, ohne ihn direkt zu übertragen.
       Ziel: Sicherer Aufbau eines gemeinsamen geheimen Schlüssels.

TLS/SSL - Transport-Sicherheit — Lösung (3/4)

3. TLS-Handshake in 4 Schritten:
    1. **ClientHello:** Client sendet unterstützte TLS-Versionen und Cipher Suites an den Server.
       Ziel: Aushandlung kompatibler Sicherheitsparameter.
    2. **ServerHello + Certificate:** Server wählt eine Cipher Suite aus und sendet sein X.509-Zertifikat.
       Ziel: Festlegung der Verschlüsselung und Authentifizierung des Servers.
    3. **Key Exchange:** Beide Seiten führen einen Schlüsselaustausch durch und berechnen einen gemeinsamen Sitzungsschlüssel, ohne ihn direkt zu übertragen.
       Ziel: Sicherer Aufbau eines gemeinsamen geheimen Schlüssels.
    4. **Finished:** Beide Seiten bestätigen den erfolgreichen und unveränderten Ablauf des Handshakes.
       Ziel: Sicherstellen der Integrität und Start der verschlüsselten Kommunikation.

TLS/SSL - Transport-Sicherheit — Lösung (4/4)

4. **Certificate Pinning:** Dabei akzeptiert eine Anwendung nur ein zuvor festgelegtes Zertifikat oder einen bestimmten Public Key, anstatt allen durch Zertifizierungsstellen signierten Zertifikaten zu vertrauen.

   Ziel: Schutz vor Man-in-the-Middle-Angriffen, selbst wenn eine Zertifizierungsstelle kompromittiert ist.

   Relevanz: Besonders wichtig ist Certificate Pinning in Mobile Apps (z.B. Android oder iOS), da diese häufig in unsicheren Netzwerken genutzt werden.

TLS/SSL - Transport-Sicherheit — Lösung (4/4)

4. **Certificate Pinning:** Dabei akzeptiert eine Anwendung nur ein zuvor festgelegtes Zertifikat oder einen bestimmten Public Key, anstatt allen durch Zertifizierungsstellen signierten Zertifikaten zu vertrauen.

   Ziel: Schutz vor Man-in-the-Middle-Angriffen, selbst wenn eine Zertifizierungsstelle kompromittiert ist.

   Relevanz: Besonders wichtig ist Certificate Pinning in Mobile Apps (z.B. Android oder iOS), da diese häufig in unsicheren Netzwerken genutzt werden.

Zusatz: SSL (Secure Sockets Layer) und TLS (Transport Layer Security) sind kryptographische Protokolle zur Absicherung der Datenübertragung im Internet. Sie verschlüsseln die Kommunikation und gewährleisten Vertraulichkeit, Integrität und Authentizität, wobei TLS der moderne und sichere Nachfolger von SSL ist.